

Engram Memory

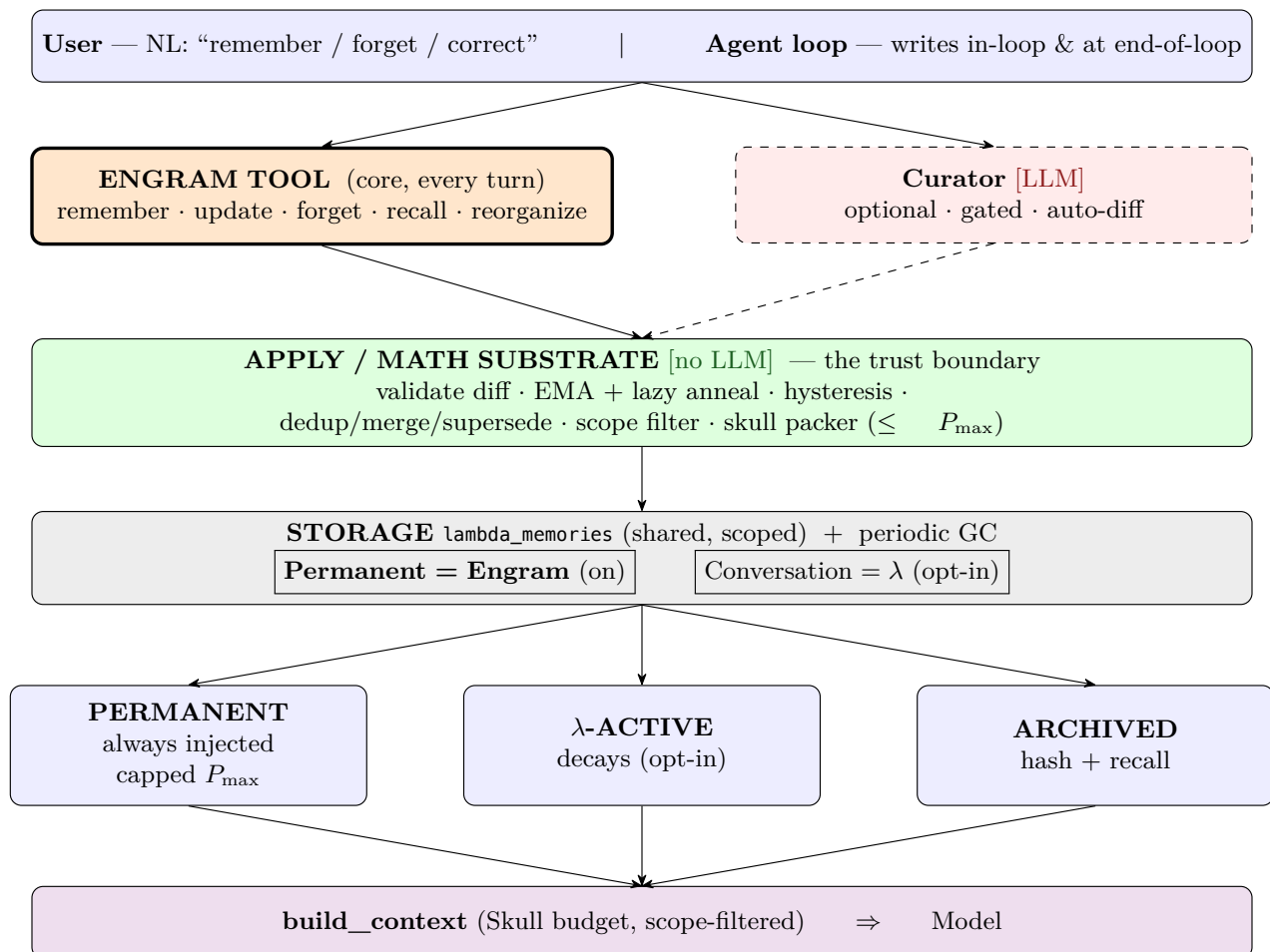
Permanent, self-curating long-term memory for TEMM1E — a core tool (v5.7.0, rev 4 — hardened)

λ -Memory forgets; Engram remembers. Default-on; when disabled, degrades to a simple capped `MEMORY.md`.

1. Requirements

1. **Core tool, NL-driven** — “hey Tem, remember my birthday” just works; no slash commands.
2. **Self-sustaining, no repetition** — the agent curates; the user never repeats a fact.
3. **Skull-bound** — never exceeds the window; permanent tier has a hard ceiling.
4. **Decay & demotion** — stale facts fade and demote automatically (even with the curator off).
5. **Immediate control** — “remember / forget / correct” applies instantly and sticks.
6. **Default-on; off $\Rightarrow$ simple `MEMORY.md`** — one flag, two paths; the fallback reuses the same tool + render, minus curator/decay/tiers.
7. **Coexists with λ -Memory** (opt-in, conversation-only today) without changing it.

2. Architecture



Legend: green = pure math (every turn, free); red dashed = the only LLM touchpoint. Writes come from the agent’s in-loop tool calls; the Curator is an optional safety-net doing the same automatically. The math layer is the *trust boundary* — it never lets the LLM violate an invariant.

3. Memory as a core tool

Slash commands are redundant; the agent calls a tool in natural language. Extends the existing `memory_manage` tool.

Action	When the agent calls it
<code>remember{content,type,scope}</code>	you say “remember...” or it learns a durable fact (in-loop / end-of-loop)
<code>update{target,content}</code>	a fact changed (“actually it’s RunPod now”) → supersede
<code>forget{target}</code>	“forget that”
<code>recall{query}</code>	needs a fact not currently in context (FTS over all tiers)

4. Coexistence with λ -Memory (no conflict)

λ -Memory today is off by default (gated by `shared_memory_strategy`; opt-in via `/memory lambda`) and stores only `Conversation` turns. Engram reuses its storage + decay/FTS code but is a separate feature: distinct `memory_type=Permanent`, its own write path (the tool), its own default-on flag (works in Echo mode), and always-on (scope-filtered) injection. `/memory echo|lambda` is unchanged.

5. Scoring & lifecycle (hardened formulas)

Each fact stores an importance I and `last_accessed`; everything else is derived lazily at read time (scores are never stored — same principle as λ).

Seed (first time a fact is judged, importance $\hat{I} \in [0, 5]$): $I \leftarrow \hat{I}$
Update (re-judged or reinforced): $I \leftarrow \text{clip}((1 - \eta)I + \eta\hat{I}, 0, 5)$, `last_accessed` ← now
Effective importance (lazy, read-time): $I_{\text{eff}} = I \cdot e^{-\Delta t / \tau}$, $\Delta t = \max(0, \text{now} - \text{last_accessed})$
User pin: $I_{\text{eff}} = 5$ always (no anneal; only the user unpins).

Tiering (hysteresis, no flapping): `pinned` $\vee I_{\text{eff}} \geq \theta_{\uparrow} \Rightarrow \text{PERMANENT}$; $(\text{PERMANENT} \wedge I_{\text{eff}} \leq \theta_{\downarrow}) \Rightarrow \text{demote to } \lambda$; $I_{\text{eff}} < \varepsilon \Rightarrow \text{archived (hash)}$.

- **“Stated once is enough”** — a confidently-judged fact ($\hat{I} \geq \theta_{\uparrow}$) is permanent on first write; an unsure one ($\hat{I}$ low) stays in λ and is promoted only if the agent keeps judging it relevant. No *user* repetition either way.
- **Forgetting is curator-independent** — the lazy anneal drags I_{eff} down with disuse, so stale facts demote by pure math even if the curator is off.
- **Recency is free** — using a fact resets `last_accessed`, so I_{eff} already encodes “recently relevant” — one constant τ , no separate recency term.

6. Skull binding (holds on every model)

$$\text{bone} + \text{active} + \text{blueprint} + \underbrace{\text{permanent}}_{\leq P_{\max}} + \underbrace{\lambda\text{-rem}}_{\text{absorbs rest}} + \text{reserve} + \text{guard} \leq \text{skull}.$$

$\text{permanent}_{\text{render}} = \min(P_{\max}, \text{free space})$, ranked by I_{eff} . If a small-context model leaves little room, permanent **degrades fidelity** (full→summary→essence), then renders only the top- I_{eff} subset *this turn* — it is never demoted for lack of space, and the window never overflows. Budgeted against $1.15 \times \text{estimate_tokens}$ for the len/4 heuristic’s error.

7. Robustness — bulletproof (failure mode → guard)

Failure mode	Guard
Curator hallucinates / malformed diff	apply layer validates schema; unknown ids rejected
Curator tries to demote/delete a user pin	pins immutable in apply layer — LLM cannot touch
Permanent set exceeds the window	cap $P_{\max}$ + fidelity degrade + drop-this-turn
Tiny-context model (bone+active large)	render top- I_{eff} subset at lowest fidelity
Stale fact never re-judged (curator off)	lazy time-anneal demotes it (no LLM)
λ store grows forever on disk	scheduled GC prunes old low- I_{eff} non-pinned
Multi-user / multi-chat leakage	scope (global/user/chat) filter at injection
Clock skew (now < last_accessed)	$\Delta t = \max(0, \cdot)$ (saturating)
Token estimate undercount	budget against $1.15 \times + 2\%$ guard band
Score thrash turn-to-turn	EMA smoothing + hysteresis band $[\theta_{\downarrow}, \theta_{\uparrow}]$
Supersession clobbers wrong fact	match on subject-key + similarity; old kept as provenance

8. Long-term behavior — timeproof

Over months/years nothing drifts or saturates: $I \in [0, 5]$ is clipped; context is bounded by $P_{\max}$; on-disk storage is bounded by GC; the anneal guarantees forgetting *without* an LLM; no counter grows unbounded (timestamps are u64 seconds). Behavior is identical across models because every budget is *Skull-relative*, not a flat number. A fact’s whole life — promote, persist, anneal, demote, recall — is one formula evaluated lazily.

9. Where the LLM is vs math

Operation	Engine	When
seed / EMA / anneal / hysteresis / tiering	[MATH]	every turn, free
budget pack, fidelity, scope filter, apply diff	[MATH]	every turn, free
dedup / merge / supersede, GC	[MATH]	deterministic / scheduled
agent tool call (remember/update/...)	[LLM]	part of the normal loop
Curator re-evaluation	[LLM]	1 bounded call / substantive run (gated, optional)

10. Cost & cadence

Three buckets; only the Curator is new vs a plain MEMORY.md.

Bucket	When	~Tokens	New vs MEMORY.md
Read (inject block)	every turn	$\leq P_{\max}$ in, cacheable	No (file injected too)
Write (tool call)	memory-write turns	+1 round-trip	No (file edit is a tool)
Curator (self-heal)	substantive runs, gated	1 call: $\sim 1\text{--}3\text{k}$ in + $\sim 150\text{--}300$ out	Yes (the only one)

Heavy user (~ 100 turns/day, $\sim 30\%$ substantive $\Rightarrow \sim 30$ curator calls/day): $\sim 1.2\text{M}$ in + 120k out / month. Illustrative: self-hosted $\approx \$0$; mid-tier $\approx \$0.8/\text{mo}$; flagship $\approx \$5/\text{mo}$. Prompt caching cuts the input further. **Cadence is the cost dial:** per-substantive-run (default) $\cdot$ every N $\cdot$ session-end $\cdot$ off. **Engram off (MEMORY.md fallback) $\Rightarrow$ Curator = $\$0$.**

11. Dry run (hardened model)

B=baseline call (with or without Engram); **+E**=Engram-added.

event	memory action	added LLM
“remember my birthday 2/3/1994”	remember $\rightarrow$ User-pin $I_{\text{eff}}=5$ (date normalized)	B (tool round-trip)
deploy task; “I use Lambda Labs” (clear pref) (if only a passing mention, $\hat{I}=2.4$)	remember $\hat{I}=4 \Rightarrow I=4 \geq \theta_{\uparrow} \rightarrow$ permanent day 0 stays λ ; promotes later only if re-judged	B + Curator $\times 1$ –
60 days, no deploy talk	anneal: $I_{\text{eff}}=4e^{-60/60}=1.5 \leq$ $\theta_{\downarrow} \rightarrow$ auto-demote (no user, no LLM)	none
“actually I moved to RunPod”	update supersede; new $\hat{I}=4.5$ $\rightarrow$ permanent now	B (tool round-trip)
new session: “when’s my birthday?”	already in Permanent block (scope=user)	B only
“what was that fix last month?”	recall $\rightarrow$ FTS hit, reheated	B (tool round-trip)

No repetition anywhere; user pin instant; promotion by the agent’s judgment; demotion by math.

12. Data model & config

```

LambdaMemoryEntry (+ fields, additive)
+ importance: f32      // I (seed=judgment, EMA on update)
+ pinned_by:  enum { None, Agent, User }    // User => I_eff=5, locked
+ scope:      enum { Global, User(id), Chat(id) } // injection filter
+ subject_key: String // supersession ("pref:gpu-provider")
+ fact_type:  enum { Identity, Preference, Project,
                    Constraint, Reference }
+ links:      Vec<Hash>
memory_type = Permanent           // distinct from Conversation (lambda)

```

```
derived lazily: I_eff, tier      // never stored

config [memory.engram]
  enabled      = true           // off => simple capped MEMORY.md
  curator      = "substantive"  // | "every:N" | "session-end" | "off"
  p_max_frac   = 0.10 ; max_facts = 32
  eta = 0.4 ; theta_up = 3.5 ; theta_down = 2.0 ; tau_days = 60
```

13. Summary

Engram is a default-on *core tool*: the agent remembers/updates/forgets in natural language; a deterministic math layer seeds, smooths (EMA), forgets (lazy anneal), bounds (Skull cap), scopes, and GCs — and is the trust boundary that the optional LLM Curator cannot violate. Promotion is the agent’s judgment (once is enough); demotion is pure math (timeproof even with the curator off); user pins are instant and immutable. It reuses λ -Memory’s engine yet coexists cleanly, and degrades to a simple capped `MEMORY.md` when disabled. Bulletproof (failure-mode guards), timeproof (clipped + bounded + Skull-relative), token-efficient, and low-touch.